

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: A. Bhanu Prasad

Roll No.: A24126552068

Date: 01-09-2026

1. TITLE

Implement Modules in NodeJS

2. AIM

To understand and demonstrate the use of Node.js built-in modules — http, os, and path — and to create and use a custom user-defined module with ES6 export and import syntax.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the CommonJS module system using require().
 - To create a basic HTTP server using the built-in http module.
 - To retrieve system and operating-system-level information using the built-in os module.
 - To manipulate and inspect file paths using the built-in path module.
 - To create a custom user-defined module and export multiple functions and a constant using ES6 export syntax.
 - To import and use functions from a user-defined module using ES6 import syntax.
 - To differentiate between the CommonJS (require/module.exports) and ES module (import/export) systems.
-

4. THEORY

Node.js Modules — Node.js organizes code into reusable units called modules. Built-in modules such as http, os, and path ship with Node and are loaded with require(); user-defined modules are files the

developer writes and exposes to other files using `export/import` (ES modules) or `module.exports/require()` (CommonJS).

http Module — provides `http.createServer()` to build a web server. The callback receives a request (`req`) and response (`res`) object for each incoming connection; `res.write()` queues data to send, `res.end()` finishes the response, and `server.listen(port)` starts listening for connections.

os Module — exposes operating-system-related utility methods such as `platform()`, `arch()`, `totalmem()`, `freemem()`, `hostname()`, `homedir()`, `tmpdir()`, `uptime()`, `userInfo()`, `type()`, and `release()`, used to query the environment Node is running in.

path Module — provides utilities for working with file and directory paths, including `join()` to combine segments, `resolve()` to produce an absolute path, `basename()/dirname()` to extract the filename or directory, `extname()` to get the file extension, and `parse()` to break a path into its component parts.

User-Defined Modules (ES6) — `export const` and `export function` expose a constant or function from a file so it can be pulled into another file with `import { name } from './file.js'`, enabling code to be organized across multiple files.

5. ALGORITHM / PROCEDURE

- Create a file named `http_module.js`; require the `http` module and use `http.createServer()` to build a server that writes "Hello World" and ends the response.
 - Call `server.listen(3000)` and log a message confirming the server is running.
 - Create a file named `os_module.js`; require the `os` module and log the result of `platform()`, `arch()`, `totalmem()`, `freemem()`, `hostname()`, `homedir()`, `tmpdir()`, `uptime()`, `userInfo()`, `type()`, and `release()`.
 - Create a file named `path_module.js`; require the `path` module and log the result of `join()`, `resolve()`, `basename()`, `dirname()`, `extname()`, and `parse()` on sample paths.
 - Create a file named `userdefined_modules.js`; declare and export a `PI` constant and the functions `add`, `multiply`, `subtract`, `division`, and `hello` using ES6 export syntax.
 - Create a file named `test.js`; import `PI` and all five functions from `userdefined_modules.js` using ES6 import syntax.
 - In `test.js`, log the value of `PI` and the results of calling `add()`, `multiply()`, `division()`, and `subtract()`, then call `hello("Bhanu")`.
 - Ensure the project is configured for ES modules ("`type`": "`module`" in `package.json`, or a `.mjs` extension) so `import/export` syntax works in `test.js` and `userdefined_modules.js`.
 - Run each file individually with `node <filename>` in the terminal and record the console output.
 - For `http_module.js`, additionally open `http://localhost:3000` in a browser to verify the server response.
 - Record expected vs. actual output for each test case.
-

6. PROGRAM

`http_module.js`

```
const http = require("http");
```

```
// Create a web server
const server = http.createServer((req, res) => {

  // Send text data to the client/browser
  res.write("Hello World");

  // End the response and send it to the client
  res.end();
});

// Start the server and listen on port 3000
server.listen(3000);

console.log("Server is running at http://localhost:3000");
```

os_module.js

```
const os = require("os");

// Returns the operating system platform
console.log("os.platform():", os.platform());

// Returns the CPU architecture
console.log("os.arch():", os.arch());

// Returns total system memory in bytes
console.log("os.totalmem():", os.totalmem());

// Returns free system memory in bytes
console.log("os.freemem():", os.freemem());

// Returns the hostname of the operating system
console.log("os.hostname():", os.hostname());

// Returns the current user's home directory
console.log("os.homedir():", os.homedir());
```

```
// Returns the default directory for temporary files
console.log("os.tmpdir():", os.tmpdir());
```

```
// Returns system uptime in seconds
console.log("os.uptime():", os.uptime());
```

```
// Returns information about the current user
console.log("os.userInfo():", os.userInfo());
```

```
// Returns the operating system name
console.log("os.type():", os.type());
```

```
// Returns the operating system release/version
console.log("os.release():", os.release());
```

path_module.js

```
const path = require("path");
```

```
// combine multiple path segments into a single file or directory path
const p = path.join("users", "Bhanu", "Documents", "Projects", "data.txt");
console.log(p);
```

```
// Creates an absolute path.
console.log(path.resolve("data.txt"));
```

```
// Gets filename.
console.log(path.basename("C:/Users/Bhanu/Documents/Projects/full-stack-web-development/Record/Week5"));
```

```
// Gets File Directory
console.log(path.dirname("C:/Users/Bhanu/Documents/Projects/full-stack-web-development/Record/Week5"));
```

```
// Gets the Extension
console.log(path.extname("image.jpg"));
```

```
// break a file path into its individual components.
```

```
console.log(path.parse("/home/user/data.txt"));
```

userdefined_modules.js

```
export const PI = 3.14159;
```

```
export function add(a, b) {  
    return a + b;  
}
```

```
export function multiply(a, b) {  
    return a * b;  
}
```

```
export function subtract(a, b) {  
    return a - b;  
}
```

```
export function division(a, b) {  
    return a/b;  
}
```

```
export function hello(s) {  
    console.log(`Hello ${s}`)  
}
```

test.js

```
import {  
    PI,  
    add,  
    division,  
    multiply,  
    subtract,  
    hello  
} from './userdefined_modules.js';
```

```
console.log(PI);
console.log(add(2, 3));
console.log(multiply(4, 5));
console.log(division(4, 2));
console.log(subtract(4, 2));

hello("Bhanu");
```

7. CODE EXPLANATION

Code	Explanation
<code>require("http")</code>	Loads Node's built-in HTTP module using CommonJS syntax.
<code>http.createServer()</code>	Creates an HTTP server with a callback that runs for each incoming request.
<code>res.write()</code>	Sends a chunk of response data to the client.
<code>res.end()</code>	Signals that the response is complete and sends it to the client.
<code>server.listen(3000)</code>	Starts the server listening for connections on port 3000.
<code>require("os")</code>	Loads Node's built-in Operating System module.
<code>os.platform() / os.arch()</code>	Return the operating system platform and CPU architecture.
<code>os.totalmem() / os.freemem()</code>	Return the total and free system memory, in bytes.
<code>os.userInfo()</code>	Returns information about the currently logged-in user.
<code>require("path")</code>	Loads Node's built-in Path module.
<code>path.join()</code>	Combines multiple path segments into one normalized path string.
<code>path.resolve()</code>	Converts a relative path into an absolute path.
<code>path.basename() / path.dirname()</code>	Extract the filename and the directory portion of a path.
<code>path.extname()</code>	Extracts the file extension from a path.
<code>path.parse()</code>	Breaks a path into an object of its root, dir, base, name, and ext parts.
<code>export const / export function</code>	Makes a constant or function available for import in other files.
<code>import { ... } from './file.js'</code>	Imports specific named exports from a user-defined module.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Run node http_module.js and open http://localhost:3000	Browser displays "Hello World"; terminal logs the running message	Server started and browser displayed Hello World	Pass
TC-02	Run node os_module.js	Terminal prints platform, architecture, memory, hostname, and related OS details	All OS details printed correctly	Pass
TC-03	Run node path_module.js — path.join() call	Terminal prints the combined, normalized path string	Joined path printed correctly	Pass
TC-04	path.resolve("data.txt") in path_module.js	Terminal prints the absolute path to data.txt	Absolute path printed correctly	Pass
TC-05	path.basename() and path.dirname() calls	Terminal prints the filename and directory separately	Filename and directory printed correctly	Pass
TC-06	path.extname("image.jpg") call	Terminal prints ".jpg"	Extension printed as ".jpg"	Pass
TC-07	path.parse() call	Terminal prints an object with root, dir, base, ext, and name	Parsed path object printed correctly	Pass
TC-08	Run node test.js	Terminal prints PI and the results of add, multiply, division, and subtract	All imported values and results printed correctly	Pass
TC-09	hello("Bhanu") call in test.js	Terminal prints "Hello Bhanu"	Greeting printed correctly	Pass

9. OUTPUT

Output 1: http_module.js — Terminal and Browser Output

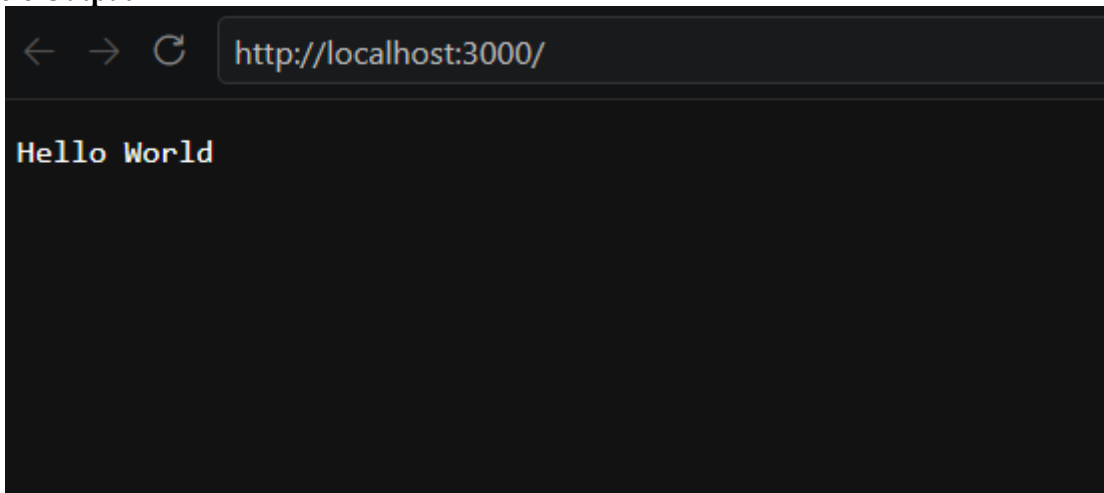
The terminal shows the "Server is running at http://localhost:3000" message, and the browser displays "Hello World" when the address is opened.

Terminal Output

```
PS C:\Users\Purna\Documents\Projects\FullStack\Records\week_5> node http_module.js
Server is running at http://localhost:3000

```

Console Output



Output 2: os_module.js — Terminal Output

The terminal prints the platform, architecture, total and free memory, hostname, home and temp directories, uptime, user info, OS type, and release.

```
● PS C:\Users\Purna\Documents\Projects\FullStack\Records\week_5> node os_module.js
os.platform(): win32
os.arch(): x64
os.totalmem(): 16471101440
os.freemem(): 6151131136
os.hostname(): DESKTOP-OS14KPM
os.homedir(): C:\Users\Purna
os.tmpdir(): C:\Users\Purna\AppData\Local\Temp
os.uptime(): 26373.546
os.userInfo(): [Object: null prototype] {
  uid: -1,
  gid: -1,
  username: 'Purna',
  homedir: 'C:\\Users\\Purna',
  shell: null
}
os.type(): Windows_NT
os.release(): 10.0.26200
○ PS C:\Users\Purna\Documents\Projects\FullStack\Records\week_5>
```

Output 3: path_module.js — Terminal Output

The terminal prints the joined path, the resolved absolute path, the basename, dirname, extension, and the parsed path object.

```
PS C:\Users\Purna\Documents\Projects\FullStack\Records\week_5> node path_module.js
users\Purna\Documents\Projects\data.txt
C:\Users\Purna\Documents\Projects\FullStack\Records\week_5\data.txt
week_5
C:/Users/Purna/Documents/Projects/FullStack/Records
.jpg
{
  root: '/',
  dir: '/home/user',
  base: 'data.txt',
  ext: '.txt',
  name: 'data'
}
```

Output 4: test.js — Terminal Output

The terminal prints the imported PI constant, the results of add, multiply, division, and subtract, and the "Hello Bhanu" greeting from the imported hello function.

```
3.14159
5
20
2
2
Hello Purna
```

10. RESULT

Thus, the Node.js built-in http, os, and path modules were successfully explored, and a custom user-defined module was created and used with ES6 export and import syntax. All nine test cases passed successfully, confirming correct server behaviour, system information retrieval, path manipulation, and module import/export functionality.